

02 - Refactorización del CRUD repetido

Ejercicio 02 — Refactorización del CRUD repetido



Foco: DRY (Don't Repeat Yourself), refactorización **Tiempo estimado:** 1–2 clases [← Volver al README](#)

Objetivo

Que uses la IA para **detectar y eliminar duplicación**. Después del ejercicio 01 tenés (al menos) 3 entidades con CRUDs casi idénticos. Si comparás `alumnos-repository.js` con `cursos-repository.js` vas a ver que cambian: el nombre de la tabla, las columnas y poco más. **El 80% es copiar y pegar.**

Acá la IA es muy buena, pero también es donde más fácil se le va la mano y te "sobre-refactoriza" algo que después no entendés.

El problema

Mirá estos dos métodos de repositorios distintos:

```
// alumnos-repository.js
getAllAsync = async () => {
  console.log(`AlumnosRepository-new.getAllAsync()`);
  const sql = `SELECT * FROM alumnos`;
  return await this.db.queryAll(sql);
}
```

```
// cursos-repository.js ← prácticamente lo mismo, cambia "alumnos" por "cursos"
getAllAsync = async () => {
  console.log(`CursosRepository.getAllAsync()`);
  const sql = `SELECT * FROM cursos`;
  return await this.db.queryAll(sql);
}
```

💡 Ojo: el `try/catch` y el manejo del `Pool` **ya** están extraídos en la clase `DbPg` (`db-pg.js`). Lo que sigue duplicado entre repositorios es el armado del SQL y el nombre de la tabla/columnas. Esa es la

duplicación que ataca este ejercicio.

Lo mismo pasa en los **services** (puro pass-through) y en los **controllers** (el patrón try/catch + status code se repite endpoint por endpoint, entidad por entidad).

Qué tenés que lograr

Elegí **una** de estas estrategias (discutila con la IA, no la elijas a ciegas).

La idea de fondo de todas es la misma: **el código común se escribe una sola vez**, y cada entidad (alumnos, cursos, materias) solo aporta **lo que tiene de distinto** (su nombre de tabla y sus columnas). Cambian en **cómo** logran eso.

A) Repository base por herencia ("es un")

Una clase "madre" `BaseRepository` tiene los métodos **comunes** (los que solo dependen de la tabla y el id: `getAllAsync`, `getByIdAsync`, `deleteByIdAsync`). Cada repositorio **hereda** de ella y solo dice su tabla + sus métodos propios.

```
// La clase madre tiene lo común, parametrizado por el nombre de la tabla
class BaseRepository {
  constructor(tabla) {
    this.tabla = tabla;
    this.db = new Db();
  }
  getAllAsync = async () => await this.db.queryAll(`SELECT * FROM ${this.tabla}`);
  // ...getByIdAsync y deleteByIdAsync también viven acá
}

// CursosRepository "ES UN" BaseRepository de la tabla cursos:
class CursosRepository extends BaseRepository {
  constructor() { super('cursos'); } // ← Le pasa su tabla a la madre

  // Solo agrega lo específico (las columnas propias del INSERT/UPDATE):
  createAsync = async (entity) => { /* INSERT INTO cursos (nombre) ... */ }
}
```

Así `CursosRepository` **ya tiene** `getAllAsync` gratis, sin volver a escribirlo. Lo mismo `AlumnosRepository` `extends BaseRepository`, pasando `'alumnos'`.

 **Palabra clave:** `extends` / **herencia**. Pensalo como "es un": un curso-repository **es un** repository base, pero de la tabla cursos.

B) Repository por composición ("tiene un"), sin herencia

En vez de heredar, cada repositorio **tiene adentro** un objeto genérico que sabe hacer lo común, y al crearlo le pasa **su nombre de tabla** por parámetro. No hay clase madre/hija: hay un objeto que se **parametriza** y al que se le **delega**.

```
// Un repositorio genérico que sabe hacer lo común para CUALQUIER tabla
class GenericRepository {
  constructor(tabla) {
    this.tabla = tabla;
    this.db = new Db();
  }
  getAllAsync = async () => await this.db.queryAll(`SELECT * FROM ${this.tabla}`);
  // ...getByIdAsync y deleteByIdAsync también viven acá
}

// CursosRepository "TIENE UN" GenericRepository configurado con 'cursos':
class CursosRepository {
  constructor() {
    this.base = new GenericRepository('cursos'); // ← crea el objeto y le pasa la tabla
  }
  // Lo común se lo PIDE a su objeto interno (delega):
  getAllAsync = async () => await this.base.getAllAsync();

  // Y agrega lo suyo:
  createAsync = async (entity) => { /* INSERT INTO cursos (nombre) ... */ }
}
```

AlumnosRepository hace lo mismo: `this.base = new GenericRepository('alumnos')` y delega lo común.

abc Diferencia con A: en A **heredás** ("es un": un curso-repo **es un** repository base). En B **tenés un** objeto adentro y lo parametrizás ("tiene un": un curso-repo **tiene un** repository genérico configurado con 'cursos'). Las dos eliminan la duplicación. Para este proyecto, **A** suele ser la más corta; **B** evita la herencia a cambio de delegar cada método común.

C) La que te proponga la IA

Cualquier otra estrategia que te ofrezca la IA, **siempre que la entiendas y la puedas justificar** en el oral. Si no la podés explicar, no la elijas.

El resultado, elijas la que elijas: **menos líneas duplicadas**, mismo comportamiento, los endpoints siguen respondiendo igual.

🔒 **Nota sobre seguridad (interpolación el nombre de tabla):** si elegís la estrategia A, el método genérico va a hacer algo como `SELECT * FROM ${this.tabla}`. Eso es **interpolación un string en el SQL** — justo lo que el ejercicio 09 te dice que NO hagas. La diferencia clave: acá `this.tabla` es una **constante que definís vos** (el desarrollador), no un dato que manda el usuario. Interpolación un valor controlado por vos es seguro; interpolación **input del usuario** (un `id`, un `nombre` que viene del `req.body`) es lo que abre la puerta a SQL injection — y eso **siempre** va con `$1`. Tenés que poder explicar esta diferencia en el oral.

🎓 Detalle para aprender: `extends` con *arrow functions* vs. métodos normales

Si elegís la **estrategia A** (herencia), vas a chocar con un detalle del **estilo del proyecto**: los métodos de los repositories están escritos como **campos de clase con arrow function** (`getAllAsync = async () => {...}`), no como métodos "normales" (`async getAllAsync() {...}`). Las dos formas heredan bien, pero **no son idénticas**.

Forma 1 — CON arrow (campos de clase), el estilo actual del proyecto:

```
// base-repository.js
export default class BaseRepository {
  constructor(tabla) { this.tabla = tabla; this.db = new Db(); }
  getAllAsync = async () => await this.db.queryAll(`SELECT * FROM ${this.tabla}`);
}

// cursos-repository.js
export default class CursosRepository extends BaseRepository {
  constructor() { super('cursos'); }
  createAsync = async (entity) => { /* INSERT INTO cursos ... */ };
}
```

Forma 2 — SIN arrow (métodos de prototipo), el estilo "tradicional":

```
// base-repository.js
export default class BaseRepository {
  constructor(tabla) { this.tabla = tabla; this.db = new Db(); }
  async getAllAsync() { return await this.db.queryAll(`SELECT * FROM ${this.tabla}`); }
}

// cursos-repository.js
export default class CursosRepository extends BaseRepository {
  constructor() { super('cursos'); }
  async createAsync(entity) { /* INSERT INTO cursos ... */ }

  // Solo con esta forma podés "extender" un método común usando super:
}
```

```

async getAllAsync() {
  const rows = await super.getAllAsync(); // ✅ corre la versión de la base...
  return rows;                          // ...y acá podrías agregarle algo
}

```

¿En qué se diferencian?

	CON arrow (metodo = async () => {})	SIN arrow (async metodo() {})
Dónde vive el método	en cada instancia (campo)	en el prototipo (compartido)
Heredar los métodos comunes	✅ funciona	✅ funciona
Llamar <code>super.getAllAsync()</code> desde el hijo	❌ no funciona (... is not a function)	✅ funciona
Pasar el método como callback suelto (<code>arr.map(repo.getAllAsync)</code>)	mantiene <code>this</code> (anda)	pierde <code>this</code> (rompe)

Qué hacer en este ejercicio

- La estrategia A solo **hereda** los comunes y **agrega** `createAsync` / `updateAsync` (sin `super.metodoComun()`), así que **las dos formas sirven**.
- Para mantener **consistencia** con el resto del proyecto, seguí con **arrow**.
- Si** necesitás extender un método común con `super.x()` , pasá **ese método puntual** a la forma sin arrow.

🤖 Pista de error típico: si "no te anda la herencia", fijate si intentaste hacer `super.getAllAsync()` sobre un método escrito como arrow-field. Esa es la trampa del estilo del proyecto.

🤖 Cómo encarar el prompting

Este ejercicio tiene **dos prompts diferenciados**:

Prompt 1 — Diagnóstico (no pidas código todavía):

Pedile a la IA que **identifique la duplicación** y te proponga **2 o 3 estrategias** de refactorización con sus pros y contras. Pegale los dos repositories. NO le pidas que refactorice aún.

Prompt 2 — Ejecución:

Una vez que elegiste una estrategia, pedile que la implemente. Acá las **restricciones** son críticas: "la API pública no debe cambiar", "los tests/Postman tienen que seguir pasando igual", "no rompas el patrón de

LogHelper ".

⚠️ **Trampa frecuente:** la IA te va a ofrecer meter un ORM (Sequelize, Prisma) para "eliminar todo el SQL repetido". Eso **no es refactorizar**, es reescribir el proyecto y cambiar el stack. Poné en las restricciones: "seguí usando `pg` con SQL crudo, no introduzcas un ORM".

💡 **Tip de iteración:** pedile a la IA que te muestre **el antes y el después en líneas de código** ("¿cuántas líneas tenía cada repository antes y cuántas comparten ahora?"). Si no bajó la duplicación, el refactor no sirvió.

🔍 Verificación del resultado

- ☐ Los 5 endpoints de `alumnos`, `cursos` y `materias` siguen respondiendo **igual que antes** (mismos status codes, mismo JSON). Probalo en Postman antes y después.
- ☐ La lógica común está **en un solo lugar** (si arreglás un bug en `getAllAsync`, se arregla para todas las entidades).
- ☐ Lo específico de cada entidad (nombre de tabla, columnas del INSERT/UPDATE) sigue siendo claro y fácil de cambiar.
- ☐ La regla de negocio de `alumnos` (calcular `edad`, validar que el curso existe) **no se perdió** en el refactor.
- ☐ No se agregó un ORM ni dependencias nuevas.

🤖 Pregunta para el oral: *si mañana agregás una tabla nueva (por ejemplo `profesores`), ¿cuántas líneas tenés que escribir ahora vs. antes del refactor? Mostrame en tu código qué heredás y qué overrideás.*

✅ Entrega

Bitácora + commit. En la reflexión, **justificá por qué elegiste esa estrategia** y qué descartaste. El "por qué no" es tan importante como el "por qué sí".